High-Performance Distributed Computing and Inference Optimizations

Sapienza University of Rome
14-15 April 2026

Sergio Orlandini
CINECA
s.orlandini@cineca.it

Tips&Tricks: SLURM Hybrid Job Example

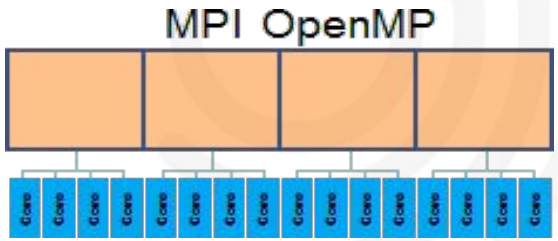
4 processes per node, each process control 1 GPU

- Use 8 cpus per task, switch the binding of the OMP threads on, and correctly map the MPI processes with the `--map-by` option

```
#SBATCH --ntasks-per-node=4        # 4 processes per node
#SBATCH --cpus-per-task=8          # 8 cores per process
#SBATCH --gres=gpu:4               # 4 gpus per task

export OMP_NUM_THREADS=8

module load openmpi
mpirun --map-by socket:PE=8 <your_exe>
```



Tips&Tricks: How to Access to a Compute Node

- You are not authorised to access compute nodes through ssh connection
- Only if a node has been reserved a compute node with SLURM scheduler, can it be accessed through ssh
- This is useful for monitoring the computational resources being used (e.g. with nvidia-smi)

NB: to access a compute node, you must exchange the ssh keys between compute and login nodes.

On a login node you must execute the following commands:

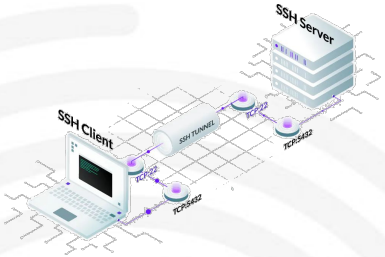
```
$ ssh-keygen -t rsa -b 4096
$ cat .ssh/id_rsa.pub > .ssh/authorized_keys
```

Tips&Tricks: Jupyter Notebook On a HPC Cluster

SSH Port Forwarding, aka SSH tunnel, allows connection from a local computer to a remote server.
A local port can be forwarded with the -L option of ssh command:

```
$ ssh -N -L LOCAL:localhost:REMOTE USERNAME@REMOTE.HOST
```

NB: -N option is used for just forwarding ports and do not execute a remote command.



- From local machine open a SSH tunnel to remote machine, Leonardo:

  ```
  $ ssh -N -L 9999:localhost:9999 USERNAME@login01-ext.leonardo.cineca.it
  ```

- Go to remote server (Leonardo) and launch jupyter notebook without open a browser:

  ```
  $ jupyter notebook --port=9999 --no-browser
  ```

  The stdout of the command should be something like:

  ```
  http://localhost:9999/?token=75f9c6d4611a636b3249cd79fe10b218ab1f1c267d4c53d1
  ```

- Open a browser and copy&paste the URL

NB: to kill process running on a port: `$ lsof -ti:9999 | xargs kill`

Tips&Tricks: Jupyter Notebook On a HPC Cluster

1. Launch jupyter notebook on remote cluster, Leonardo:

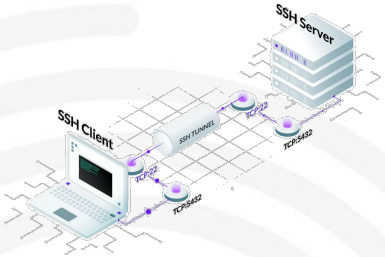
```
$ jupyter notebook --port=9999 --no-browser
```

2. Open ssh tunnel from local machine, your laptop:

```
$ ssh -N -L 9999:localhost:9999
USERNAME@login01-ext.leonardo.cineca.it
```

3. Open notebook from a browser

```
http://localhost:9999/?token=75f9c6d4611a636b3249cd79fe10b218ab1f1c267d4c53d1
```



Tips&Tricks: Jupyter Notebook On a HPC Cluster

To open a notebook directly on a compute node a double ssh tunnel is required, aka double pike with 1 ½ somersaults and half-twist.

1. Submit an interactive job on Leonardo:

```
$ srun -N 2 --ntasks-per-node=4 --gres=gpu:4 … --pty /bin/bash

lrdn2529 $  jupyter notebook --port=9999 --no-browser
```

2. Open ssh tunnel from local machine to login node and from login node to compute node:

```
$ ssh -L 9999:localhost:9999 USERNAME@login01-ext.leonardo.cineca.it ssh -L 9999:localhost:9999 -N
lrdn2529
```

3. Open notebook from a browser

```
http://localhost:9999/?token=75f9c6d4611a636b3249cd79fe10b218ab1f1c267d4c53d1
```

Tips&Tricks: Jupyter Notebook On a HPC Cluster

1. Submit an interactive job on Leonardo:

```
$ salloc -N 2 --ntasks-per-node=4 --gres=gpu:4 …
```

2. Launch jupyter notebook on remote cluster, Leonardo:

```
$ jupyter notebook --port=9999 --no-browser
```

3. Open ssh tunnel from local machine, your laptop:

```
$ ssh -N -L 9999:localhost:9999 USERNAME@login01-ext.leonardo.cineca.it
ssh -L 9999:localhost:9999 -N lrdn2529
```

4. Open notebook from a browser

```
http://localhost:9999/?token=75f9c6d4611a636b3249cd79fe10b218ab1f1c267d4c53d1
```

Tips&Tricks: How to open `tensorboard` on a cluster

1. Open ssh tunnel from local machine, your laptop:

```
$ ssh -N -L 9999:localhost:9999
USERNAME@login01-ext.leonardo.cineca.it
```

2. Open tensorboard on remote host:

```
$ tensorboard --logdir=tensorboard  --port=9999
```

3. Open a browser to your local machine and got to address:

```
http://localhost:9999
```

Tips&Tricks: torch.distributed for any Launcher

```python
def get_resources():

    if os.environ.get("RANK"):
        # launched with torchrun (python -m torch.distributed.run)
        rank = int(os.getenv("RANK"))
        local_rank = int(os.getenv("LOCAL_RANK"))
        world_size = int(os.getenv("WORLD_SIZE"))
        local_size = int(os.getenv("LOCAL_WORLD_SIZE"))

    elif os.environ.get("OMPI_COMMAND"):
        # launched with mpirun
        rank = int(os.environ["OMPI_COMM_WORLD_RANK"])
        local_rank = int(os.environ["OMPI_COMM_WORLD_LOCAL_RANK"])
        world_size = int(os.environ["OMPI_COMM_WORLD_SIZE"])
        local_size = int(os.environ["OMPI_COMM_WORLD_LOCAL_SIZE"])

    else:
        # launched with srun (SLURM)
        rank = int(os.environ["SLURM_PROCID"])
        local_rank = int(os.environ["SLURM_LOCALID"])
        world_size = int(os.environ["SLURM_NPROCS"])
        local_size = int(os.environ["SLURM_NTASKS_PER_NODE"])

    num_workers = int(os.environ["SLURM_CPUS_PER_TASK"])

    return rank, local_rank, world_size, local_size, num_workers

rank, local_rank, world_size, local_size, num_workers = get_resources()

torch.distributed.init_process_group(backend="nccl", rank=rank, world_size=world_size)

torch.cuda.set_device(local_rank)
```

Tips&Tricks: Python Virtual Environment

```
# load cineca-ai environment
module load profile/candidate
module load cineca-ai/4.1.1

# create your python environment
python -m venv --system-site-packages myenv

# load your environment
source myenv/bin/activate

# install jupyter notebook
pip install notebook
pip install matplotlib
```

DL @LEONARDO - SLURM

```
#!/bin/bash
#SBATCH -N 1                          # number of nodes
#SBATCH --ntasks-per-node=1           # number of processes per compute node
#SBATCH --cpus-per-task=8             # number of cores per process
#SBATCH --gres=gpu:1                  # number of gpus per node
#SBATCH -t 1:00:00                    # total run time of the job allocation
#SBATCH -o job.out                    # for stdout redirection
#SBATCH -e job.err                    # for stderr redirection
#SBATCH -p boost_usr_prod             # partition for resource allocation
#SBATCH -A tra26_sapai                # account name for allocation

module load profile/deeplrn
module load cineca-ai/4.1.1

...
```

DL @LEONARDO - SLURM

```
$ srun -N 1 --ntasks-per-node=1 --cpus-per-task=8 --gres=gpu:1 -p
boost_usr_prod -A tra26_sapai --pty /bin/bash

srun: job 1419573 queued and waiting for resources
srun: job 1419573 has been allocated resources

lrdn1065 $ hostname
lrdn1065.leonardo.local

$ echo $SLURM_NODELIST
lrdn[1065,1071]
```

DL @LEONARDO - SLURM

```
$ salloc -N 1 --ntasks-per-node=1 --cpus-per-task=8 --gres=gpu:1 -p
boost_usr_prod -A tra26_sapai

salloc: Pending job allocation 1419565
salloc: job 1419565 queued and waiting for resources
salloc: job 1419565 has been allocated resources
salloc: Granted job allocation 1419565
salloc: Waiting for resource configuration
salloc: Nodes lrdn[2361-2362] are ready for job

login01 $ hostname
login01.leonardo.local

$ echo $SLURM_NODELIST
lrdn[2361-2362]
```

DL @LEONARDO - SLURM

```
$ mpirun --report-bindings --map-by socket:PE=8 ./src/mind-the-bind/mind-the-bind.x

Number of nodes ............. 1
Hosts ....................... lrdn1262
Number of processes ......... 4  / SLURM_NPROCS
Number of cpus per task ..... 8  / SLURM_CPUS_PER_TASK
OMP_NUM_THREADS ............. 8

[lrdn1262:1494812] rank 0: [B/B/B/B/B/B/B/B/./././././././././././././././././././././././././././.]
[lrdn1262:1494812] rank 1: [./././././././././B/B/B/B/B/B/B/B/././././././././././././././././././.]
[lrdn1262:1494812] rank 2: [././././././././././././././././././B/B/B/B/B/B/B/B/././././././././.]
[lrdn1262:1494812] rank 3: [././././././././././././././././././././././././././././././B/B/B/B/B/B/B/B]

mpi_rank  0, thread_id  0 ( 0/ 0) on node lrdn1262.leonardo.local, core 0
mpi_rank  0, thread_id  1 ( 0/ 1) on node lrdn1262.leonardo.local, core 1
mpi_rank  0, thread_id  2 ( 0/ 2) on node lrdn1262.leonardo.local, core 2
mpi_rank  0, thread_id  3 ( 0/ 3) on node lrdn1262.leonardo.local, core 3
mpi_rank  0, thread_id  4 ( 0/ 4) on node lrdn1262.leonardo.local, core 4
mpi_rank  0, thread_id  5 ( 0/ 5) on node lrdn1262.leonardo.local, core 5
mpi_rank  0, thread_id  6 ( 0/ 6) on node lrdn1262.leonardo.local, core 6
mpi_rank  0, thread_id  7 ( 0/ 7) on node lrdn1262.leonardo.local, core 7
mpi_rank  1, thread_id  0 ( 1/ 0) on node lrdn1262.leonardo.local, core 8
mpi_rank  1, thread_id  1 ( 1/ 1) on node lrdn1262.leonardo.local, core 9
mpi_rank  1, thread_id  2 ( 1/ 2) on node lrdn1262.leonardo.local, core 10
mpi_rank  1, thread_id  3 ( 1/ 3) on node lrdn1262.leonardo.local, core 11
mpi_rank  1, thread_id  4 ( 1/ 4) on node lrdn1262.leonardo.local, core 12
...
```